Le Coffre

Cette semaine, c'est vous le leak

Introduction

Cette semaine se déroule dans l'ordre inverse de l'habitude : **on attaque d'abord, on construit ensuite**.

La raison est simple. Un développeur à qui on explique la sécurité en diapositives retient des règles. Un développeur qui a réussi à supprimer les données d'une application depuis la barre d'adresse de son navigateur retient une expérience. La deuxième s'oublie beaucoup moins vite.

Lundi et mardi, vous attaquez une application volontairement vulnérable, maintenue par la fondation OWASP, et vous documentez ce que vous trouvez.

De mercredi à vendredi, vous construisez une application dont le métier est précisément de garder des secrets, en appliquant tout ce que vous venez de casser chez les autres.

Vendredi après-midi, un autre binôme essaie de casser la vôtre.

Cadre déontologique

À lire avant tout le reste, et il n'est pas négociable.

Vos tests se font **exclusivement** sur les deux cibles autorisées : l'instance Juice Shop que vous avez lancée sur votre propre machine, et l'application qui vous est attribuée lors de la séance de vendredi. Rien d'autre. Aucun site, aucun service, aucune infrastructure de l'école.

Toute action hors périmètre est éliminatoire sur le projet, indépendamment de la qualité du reste.

Ce n'est pas une règle scolaire. Dans la vie professionnelle, un test d'intrusion sans autorisation écrite préalable est un délit, et la bonne intention ne constitue pas une défense. Les compétences que vous acquérez cette semaine s'exercent dans un cadre contractuel, toujours.

Phase 1 : attaquer

La cible

OWASP Juice Shop, application délibérément vulnérable, gratuite, maintenue par la fondation OWASP et utilisée dans le monde entier pour la formation. Elle couvre l'intégralité du Top 10 et propose des défis de difficulté croissante.

Elle se lance en une commande, donnée au tableau lors du kick-off. Vous n'avez pas besoin de comprendre l'outil qui la lance : la conteneurisation est une thématique du printemps, pas de cette semaine. Vous n'avez pas non plus à lire ni à modifier son code. C'est une boîte noire, vous l'attaquez de l'extérieur.

Sa documentation est en anglais. C'est volontaire, la lecture de documentation technique en anglais fait partie des attendus du titre.

Ce qui est attendu

Un rapport d'attaque contenant **au moins six failles**, appartenant à six catégories différentes du Top 10.

Pour chacune :

- la catégorie OWASP concernée ;
- la démarche suivie pour la trouver ;
- **la preuve d'exploitation** : la requête envoyée, la réponse obtenue, une capture si nécessaire ;
- l'impact concret pour l'exploitant de l'application, formulé en langage clair. Pas « injection SQL possible » mais « n'importe quel visiteur peut lire l'intégralité des comptes clients, mots de passe hachés compris ».

Au moins trois de ces six failles doivent avoir été trouvées à la main, sans outil automatique. Un scan avec OWASP ZAP en mode baseline est autorisé et même encouragé en complément, mais un scanner ne trouve jamais un défaut de logique métier, et c'est précisément là que se situent les failles intéressantes.

Phase 2 : construire

Le produit

Une API de **gestion de secrets d'équipe**. Des utilisateurs y déposent des identifiants techniques, en partagent certains avec des collègues, et tout accès est tracé.

Le domaine est minuscule, et c'est voulu : chaque exigence fonctionnelle est en réalité une exigence de sécurité.

Le modèle minimal

Objet	Rôle
User	un utilisateur, avec ses identifiants de connexion et son rôle
Secret	un secret : libellé, valeur, propriétaire
Share	le partage d'un secret vers un autre utilisateur, en lecture
AccessLog	la trace de chaque consultation d'un secret

Rappel : **le code s'écrit en anglais**, entités, champs, routes et énumérations comprises. Le français reste pour les messages d'interface.

Les exigences de sécurité, qui sont le sujet

Authentification

- Mots de passe **hachés** avec bcrypt ou argon2. Jamais en clair, jamais chiffrés de façon réversible, et vous devez savoir expliquer la différence entre hacher et chiffrer.
- Session ou JWT : vous choisissez, et vous **justifiez par écrit**. La question qui départage les deux réponses est celle de la révocation : que se passe-t-il quand vous voulez déconnecter quelqu'un immédiatement ?
- Politique de mot de passe appliquée côté serveur.

Autorisation

Deux mécanismes distincts, et le second est celui qu'on oublie systématiquement :

- **par rôle** : un ADMIN peut gérer les utilisateurs, un MEMBER non ;
- **par propriété de la ressource** : un utilisateur ne peut lire que les secrets dont il est propriétaire ou qui lui ont été partagés. Cette vérification se fait **côté serveur, à chaque appel**. Masquer un bouton dans une interface n'est pas une protection.

Configuration

- Aucun secret applicatif dans le code ni dans l'historique Git. Variables d'environnement, et .gitignore correct.
- CORS restreint aux origines nécessaires.
- En-têtes de sécurité posés.
- Aucune stack trace renvoyée au client, aucun message d'erreur qui révèle si un compte existe ou non.

Traçabilité

Chaque consultation d'un secret produit une entrée dans le journal d'accès. Un utilisateur ne peut ni modifier ni supprimer ce journal, y compris celui qui le concerne.

Phase 3 : la vérification croisée

Vendredi après-midi. Vous recevez l'API produite par un autre binôme, et vous disposez de deux heures pour tenter de la casser.

Grille imposée, dérivée du Top 10 :

1. Peut-on lire un secret dont on n'est ni propriétaire ni destinataire ?
2. Peut-on modifier ou supprimer une ressource appartenant à quelqu'un d'autre ?
3. Peut-on obtenir un accès sans authentification valide ?
4. Peut-on falsifier ou rejouer un jeton ?
5. Un message d'erreur révèle-t-il l'existence d'un compte, la structure de la base, ou une trace technique ?
6. Un secret applicatif est-il présent dans le dépôt ou dans son historique ?
7. Le journal d'accès est-il altérable ?

Vous rendez un rapport court à l'équipe auditée, qui répond par écrit à chaque point : corrigé, ou non applicable avec argument.

Livrables

1. Rapport d'attaque Juice Shop : six failles minimum sur six catégories, dont trois trouvées sans outil automatique, chacune avec sa preuve et son impact.
2. API de gestion de secrets, sécurisée selon les exigences ci-dessus.
3. Note de justification des choix d'authentification et d'autorisation, 1 page.
4. Rapport de vérification croisée produit sur l'API partenaire.

5. Réponse écrite au rapport reçu sur votre propre API.
6. README avec instructions de lancement et gestion des variables d'environnement.

Pour aller plus loin…

Le secret chiffré au repos.

Dans votre API, la valeur d'un secret est stockée en clair : un dump de la base la révèle intégralement, et le journal d'accès n'y change rien. Chiffrez-la, avec une clé qui ne vit pas dans la base. C'est l'occasion de vérifier que vous avez compris la distinction que le sujet vous demande de savoir expliquer : un mot de passe se hache parce qu'on n'a jamais besoin de le relire, un secret d'équipe se chiffre parce qu'on doit le rendre. Puis répondez par écrit à la question qui fait mal : comment change-t-on cette clé sans perdre les secrets existants ?

La révocation, pour de vrai.

Le sujet vous fait justifier par écrit votre choix entre session et JWT en pointant la révocation. Implémentez-la. Selon votre choix, ce sera une invalidation de session, une liste de révocation, ou des jetons courts adossés à un jeton de renouvellement. Puis démontrez-la : un compte compromis, une action d'administrateur, et l'appel suivant échoue immédiatement.

Contre l'énumération de comptes.

Posez une limitation de débit sur la connexion, et faites en sorte qu'un compte inexistant et un mot de passe faux produisent la même réponse et le même temps de réponse. Ce second point est celui qu'on rate : vérifier un mot de passe haché est lent, ne trouver aucun utilisateur est instantané, et cet écart mesurable suffit à énumérer vos comptes. Chronométrez avant, chronométrez après, joignez les chiffres.

Compétences visées

- **BC01-C3 :** développer des composants métier
- **BC02-C2 :** définir l'architecture logicielle d'une application
- **BC02-C4 :** développer des composants d'accès aux données, volet sécurité

Rendu

Le projet est à rendre sur https://github.com/prenom-nom/LeCoffre
Pensez à donner les droits sur le répertoire à **deepthoughtlaplateforme** !

Structure attendue :

```
LeCoffre/
  src/            (API Spring Boot)
  docs/           (rapport Juice Shop, note de justification, rapports croisés)
  .env.example    (variables attendues, sans aucune valeur réelle)
  README.md
```

Base de connaissances

OWASP

- OWASP Top 10
- OWASP Juice Shop projet
- Juice Shop, guide compagnon en anglais
- OWASP ZAP
- Cheat sheet, stockage des mots de passe
- Cheat sheet, autorisation

Spring Security

- Documentation de référence
- Authentification par mot de passe
- Autorisation par méthode
- CORS

Transverse

- MDN, en-têtes de sécurité HTTP
- Variables d'environnement avec Spring Boot